Asistente RAG con Solr y Milvus sobre el corpus dragons_rag

Recuperación léxica y densa con modelos de lenguaje locales

Grupo 4

Massimo Enrique Maimone Guazzo Jose Miguel Bejarano Restrepo
Juan Felipe Guzmán Sandoval Silvio Mauricio Morales Osorio
David Alejandro Castillo Chiquiza

Docente: Ing. Luis Gabriel Moreno Sandoval, PhD
Procesamiento de Lenguaje Natural
Pontificia Universidad Javeriana, Bogotá

Noviembre de 2026

Resumen

Se desarrolló un asistente de preguntas y respuestas con recuperación aumentada (RAG) sobre el corpus dragons_rag, con 154.477 textos únicos de noticias y publicaciones de Bluesky sobre clima y política ambiental, en español y en inglés. Solr aporta la línea base léxica (BM25) y Milvus la recuperación densa con multilingual-e5-small. Cuatro modelos locales de Ollama reciben la misma evidencia, y la evaluación usa 48 preguntas e intervalos de confianza bootstrap del 95 %, en un portátil con una sola GPU.

Milvus recuperó mejor que Solr en el piloto, con un Recall@5 de 0,946 frente a 0,767 en las 40 preguntas con respuesta, pero con solo 24 preguntas de prueba esa ventaja no resulta concluyente en las respuestas. Lo que más separó la calidad fue el modelo: gemma2:2b cita un documento relevante en el 95 % de sus respuestas con Milvus, mientras que los modelos de 1B y 1,5B casi nunca citan. Con 10.000 documentos el Recall@5 de Milvus baja a 0,700. Ninguno de los 48 intentos de inyección tuvo éxito, aunque la cifra no permite acotar la tasa real. Dos evaluadores humanos coincidieron con un kappa ponderado de 0,60 a 0,78.

Índice

1. Problema, pregunta de investigación e hipótesis **2**

2. Datos y perfilado **3**
2.1. Corpus, duplicados y sesgos 3
2.2. Muestras y privacidad 3

3. Arquitectura y decisiones **4**

4. Segmentación y embeddings **5**

5. Almacenamiento e indexación **6**

6. Orquestación RAG **7**

7. API, interfaz y operación **8**

8. Método de evaluación **8**
8.1. Gold 8
8.2. Métricas y protocolo 10
8.3. Amenazas a la validez 11

9. Resultados de recuperación **11**
9.1. Ablaciones 12
9.2. Muestra de 10.000 documentos 13

10. Resultados de generación **14**
10.1. Experimento principal 14
10.2. Juez local 15
10.3. Ablaciones de generación 16
10.4. Evaluación humana 16
10.5. Comportamiento en casos especiales 17

11. Rendimiento **18**

12. Análisis de errores **19**

13. Limitaciones **20**

14. Conclusiones y recomendaciones **20**

15. Licencias, seguridad y nota sobre el corpus **21**

A. Repositorio y artefactos **22**

1. Problema, pregunta de investigación e hipótesis

El taller propone desarrollar un asistente RAG reproducible que compare modelos de lenguaje locales con la misma evidencia, sin servicios de pago y sobre un corpus real, ruidoso y multilingüe. Un sistema RAG puede presentar errores en tres etapas. La recuperación puede no traer el pasaje correcto [1]; el contexto puede presentarse de una forma que el modelo no aprovecha [2]; y el modelo puede afirmar algo que la evidencia no sostiene. Por esa razón se evaluó cada etapa por separado, y la recuperación se midió como se hace en los estudios de referencia sobre recuperación [3].

Con la misma colección, la misma segmentación y el mismo presupuesto de contexto, ¿cómo cambian la calidad de la recuperación y la fidelidad de las respuestas según el motor, ya sea BM25 en Solr o vectores en Milvus, y según el modelo de lenguaje local que redacta?

Se plantearon dos hipótesis. La primera, H1, sostiene que, con la misma evidencia, llama3.2:3b supera en fidelidad y en abstención correcta a los modelos de hasta 2B parámetros, aunque tarde al menos 1,5 veces más en generar en el percentil 95, es decir, en el tiempo por debajo del cual se genera el 95 % de las respuestas. En esta hipótesis gemma2:2b cuenta como modelo de 2B porque Ollama lo etiqueta así, aunque tiene 2,6B parámetros reales. La segunda, H2, sostiene que Milvus supera a Solr en Recall@5 cuando la pregunta está en otro idioma o parafraseada respecto del documento, y que Solr iguala o supera a Milvus cuando la pregunta gira en torno a un nombre propio.

Cada métrica lleva un intervalo de confianza bootstrap del 95 %, calculado con 1.000 remuestreos y semilla 42; en generación se remuestrean las preguntas. Se rechaza una hipótesis si la diferencia va en sentido contrario y su intervalo no incluye el cero; si los intervalos se solapan, el resultado no es concluyente. El conjunto de referencia anotado a mano, o gold, tiene 48 preguntas, de las cuales 24 son de prueba, por lo cual se anticipaban intervalos amplios.

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020

[2] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024. Versión preliminar: arXiv:2307.03172, 2023

[3] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021

2. Datos y perfilado

2.1. Corpus, duplicados y sesgos

El corpus es dragons_rag, un Parquet con 169.248 filas y 33 campos, y su integridad se verificó con la suma de verificación publicada. Los identificadores son únicos, no hay textos vacíos y los campos calculados, como la suma de control del texto y el conteo de palabras y de caracteres, se recalcularon fila por fila sin discrepancias. Hay 154.477 textos únicos; las 14.771 filas restantes repiten un texto ya visto con otra procedencia, por lo que se deduplicó conservando todas las procedencias, como se explica en la sección de segmentación.

Las dos colecciones difieren en un orden de magnitud de longitud, como muestra el Cuadro 1. La colección de páginas web, con 104.578 filas, reúne artículos de prensa, y la colección de publicaciones de Bluesky, con 64.670 filas, textos cortos. Por idioma, según el metadato publicado y sin validación independiente, el inglés es el 48,85 % de las filas, el español el 39,53 % y el 7,76 % no tiene idioma. Las fechas de creación van de 2006 a septiembre de 2026, y 23 no pudieron interpretarse. Los 20 dominios principales concentran el 80,63 % de las URL.

Cuadro 1: Longitud por colección, en palabras separadas por espacios.

Colección	Mínimo	Mediana	Media	P95	Máximo
Bluesky	1	35	32,7	52	67
Web	2	622	745,8	1.615	58.181

La recolección se guió por términos de búsqueda, y los campos que la describen indican cómo se capturó cada documento; no son etiquetas temáticas validadas, por lo que no se usaron como verdad de referencia. La cobertura depende de las fuentes disponibles y no representa a una población: Media Cloud aporta el 61,08 % de las filas, GDELT el 0,71 % y Bluesky el 38,21 %. Las conclusiones valen, por tanto, para este corpus.

2.2. Muestras y privacidad

Se trabajó con dos muestras anidadas, una piloto de 700 documentos dentro de otra de 10.000, tomadas del universo deduplicado de 154.477 documentos con muestreo estratificado proporcional por colección y semilla 42. La piloto, con 445 artículos de prensa y 255 publicaciones de Bluesky, concentra el gold y los experimentos con modelos de lenguaje; la de 10.000 documentos sirve para indexar y medir la recuperación a mayor escala. Bluesky es el 36,43 % de la muestra de 10.000 frente al 38,21 % del corpus bruto.

Los campos de autor no se publican ni se usan para recuperar. Antes de construir el contexto, los nombres de usuario de Bluesky se reemplazan por un texto genérico, y el gold guarda citas de hasta 300 caracteres con los identificadores enmascarados. No se publican el Parquet, el texto completo del corpus ni los pesos de los modelos.

Antes de esta implementación se hizo un perfilado exploratorio del mismo Parquet en cuadernos aparte, y sus cifras coinciden con las de este informe: 169.248 filas, 154.477 textos únicos y 4.434 grupos de texto idéntico. Allí también apareció el caso que más influyó en el diseño. Un prototipo con un índice de prueba de cinco documentos produjo una respuesta con las citas [1], [2], [3] y [5] sin que el contexto las sustentara. Este caso motivó el presupuesto de contexto con fragmentos enteros y la validación automática de citas, que se describen en la sección de orquestación.

3. Arquitectura y decisiones

El sistema se divide en etapas independientes: ingesta, segmentación, generación de embeddings, almacenamiento, recuperación, generación, interfaz y evaluación. Esto permite cambiar un componente en cada experimento. La Figura 1 muestra el flujo.

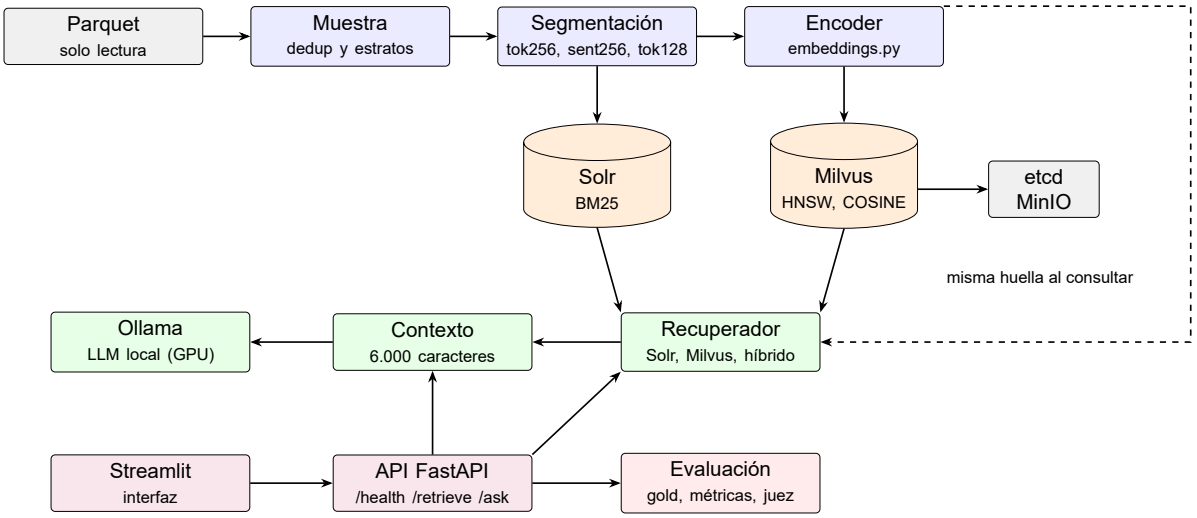


Figura 1: Arquitectura del sistema. La línea punteada indica que consulta e indexación usan el mismo encoder.

Solr es la línea base léxica, con BM25, y no tiene campos vectoriales. Milvus [4], en modo independiente con etcd y MinIO, es el único motor vectorial. Así se comparan dos enfoques de recuperación distintos. Como condición adicional se probó un híbrido que combina los 20 primeros resultados de cada motor con RRF [5], una fusión que ordena según la posición de cada documento en los dos rankings (k = 60).

El encoder, es decir, el modelo que convierte el texto en vectores, está en un único módulo, usado tanto por la ingesta como por la API, en la misma imagen y con las mismas versiones. Para que una consulta no se codifique con un modelo distinto al de los pasajes, cada colección de Milvus guarda en su descripción un identificador de

[4] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021

[5] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009

la versión del encoder, con el modelo, la revisión, los prefijos, la normalización y la dimensión. La API lo compara al consultar y, si difiere, responde con un error 503 y un mensaje en español.

Un solo archivo de Docker Compose define siete servicios: etcd, MinIO, Milvus, Solr, Ollama con GPU, la API en FastAPI y el frontend en Streamlit. Las imágenes tienen versión fija, los servicios tienen límites de memoria y CPU, y se incluyen comprobaciones de salud y registros acotados. Los puertos se publican solo en la máquina local y los datos persisten en volúmenes con nombre.

4. Segmentación y embeddings

Se implementaron tres estrategias de segmentación, medidas en tokens del tokenizador del encoder principal, es decir, en las piezas en que el modelo divide el texto. La primera, que en los cuadros aparece como “Ventanas 256”, usa ventanas de 256 tokens con un solape de 32. La segunda, “Oraciones 256”, agrupa oraciones y párrafos hasta 256 tokens sin partir ninguna oración. La tercera, “Ventanas 128”, usa ventanas de 128 tokens con solape de 16 y sirve para la ablación de tamaño y para aislar el efecto del encoder. Cada fragmento conserva sus posiciones de inicio y fin sobre el texto original sin normalizar, y no hubo desajustes de posición en ninguna configuración. El identificador de cada fragmento se deriva del documento, la estrategia y las posiciones, por lo que reindexar produce los mismos identificadores. Los fragmentos con texto normalizado idéntico en documentos distintos se fusionan en uno, que conserva la lista de apariciones y la unión de procedencias.

En el piloto, que tiene 704 documentos porque las muestras incluyen cuatro sintéticos de inyección, las ventanas de 256 tokens producen 2.499 fragmentos tras fusionar 31 duplicados, el segmentado por oraciones 2.443 y las ventanas de 128 tokens 4.540. En la muestra de 10.000 documentos, las ventanas de 256 tokens producen 36.173 fragmentos. Las publicaciones de Bluesky dan siempre un fragmento y los artículos de prensa 5,1 en promedio en el piloto y 5,2 en la muestra grande.

Se compararon dos encoders multilingües que producen vectores de 384 dimensiones. El principal es intfloat/multilingual-e5-small [6], que exige anteponer a las consultas y a los pasajes unos prefijos de texto indicados por sus autores. El de ablación es paraphrase-multilingual-MiniLM-L12-v2 [7], entrenado para medir similitud entre oraciones y sin prefijos. Ambos se usan con vectores normalizados y con distancia coseno en Milvus. Se eligió E5 porque el corpus mezcla español e inglés, las preguntas pueden estar en otro idioma que el documento y E5 se entrenó para recuperar pasajes a partir de preguntas; además, su tamaño permite ejecutarlo en CPU. MiniLM sirve para

[6] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022

[7] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019

Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020

medir cuánto aporta ese entrenamiento. MiniLM trunca su entrada a 128 tokens, así que con fragmentos de 256 la segunda mitad no entra en el vector. Por eso también se probó con ventanas de 128 tokens, como se verá en los resultados de recuperación. Como los fragmentos se miden con el tokenizador de E5, uno de 128 tokens de E5 puede superar levemente los 128 de MiniLM, y esto se declara como amenaza a la validez.

Se inspeccionaron los cinco vecinos más cercanos de ocho consultas temáticas, cuatro en cada idioma. Con E5, las consultas concretas devuelven vecinos coherentes: la de la COP16 trae cuatro de cinco textos sobre esa cumbre. En cambio, “Elecciones presidenciales en Colombia” recupera cotizaciones del dólar en Colombia, porque comparten la entidad geográfica más que el tema. Además, varios de los cinco primeros son fragmentos del mismo documento, por lo cual se midió la duplicación y se probó un contexto con un fragmento por documento. MiniLM se apoya más en la coincidencia de palabras y mezcla idiomas: la consulta de la COP16 devuelve un texto en sueco.

Con 18 pares de oraciones, seis paráfrasis, seis traducciones y seis sin relación, E5 da una similitud media de 0,951 en paráfrasis, 0,922 en traducciones y 0,772 en pares sin relación, una brecha de 0,165 entre los pares relacionados y los que no lo están; MiniLM separa mucho más, con una brecha de 0,949. Los cosenos absolutos de E5 se mueven entre 0,7 y 0,98, por lo que un umbral fijo no sirve como criterio de abstención y solo es utilizable el orden relativo. Aun así, que la separación sea mayor no significó que recuperara mejor, porque la ablación de recuperación favorece a E5. El solapamiento de los diez primeros resultados entre los dos encoders es bajo, con un índice de Jaccard medio de 0,160 sobre documentos, así que el encoder modifica el contexto que ve el modelo, además del orden.

5. Almacenamiento e indexación

En Milvus hay una colección por cada pareja de encoder y segmentación, con campos de metadatos y texto, el vector de 384 dimensiones y un índice HNSW [8], que encuentra vecinos cercanos sin comparar contra todos los vectores, con los parámetros M = 16, efConstruction = 200 y ef = 64. La escritura actualiza o inserta por identificador de fragmento, de modo que reindexar es idempotente, es decir, repetirlo no cambia el resultado. En Solr, un solo núcleo aloja las tres segmentaciones, que se distinguen con un filtro sobre el campo de estrategia. El texto se copia a campos con analizadores en español y en inglés, y las consultas usan BM25 [9]; además, la entrada del usuario se neutraliza para que sus operadores especiales no se interpreten.

Se verificó que reindexar con reanudación y reiniciar Milvus y Solr no cambia los conteos de fragmentos, que se mantuvieron en 2.499, 2.443, 4.540 y 36.173. En el piloto, Milvus indexó las ventanas de 256 tokens en 300 s, unos 8 fragmentos por

[8] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. Versión preliminar: arXiv:1603.09320, 2016/2018

[9] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009

segundo con 3 CPU. Con 10.000 documentos, Solr indexó 36.173 fragmentos en 72 s y Milvus en 4.288 s, un tiempo dominado por el encoder en CPU.

No se indexó el corpus completo. Por extrapolación, los 154.477 textos únicos darían unos 558.000 fragmentos, lo que equivale a unas 18,5 horas solo de embeddings en 4 CPU virtuales y no era posible en el tiempo disponible. El cuello de botella está en el encoder en CPU, porque la GPU se reserva para el modelo de lenguaje; el almacenamiento no limita.

6. Orquestación RAG

La recuperación devuelve fragmentos con su rango y metadatos; la generación construye el contexto, llama al modelo de lenguaje y valida las citas. En los experimentos la evidencia se congeló: se recupera una vez por pregunta y motor, y los cuatro modelos reciben exactamente el mismo contexto. Los fragmentos entran enteros y en orden de rango hasta un máximo de 6.000 caracteres. Si uno no cabe, se omite sin truncarlo, y la numeración [1..n] cubre solo los incluidos, De este modo, la evidencia que se muestra es la misma que recibió el modelo. Esta decisión responde al caso del prototipo de cinco documentos.

Los prompts son archivos con versión, identificados como v1, v2 y v3. El v1 era una orden breve de responder con la evidencia, citar con [n] y abstenerse con una frase literal; produjo abstenciones incluso cuando había evidencia suficiente, pues qwen2.5:1.5b se abstuvo en las cuatro preguntas de control aun con la evidencia exacta en [1]. El v2 dio permiso explícito de responder en parte. Con ello se corrigió ese problema, pero qwen2.5:1.5b empezó a devolver solo “[1][2][3]” y llama3.2:3b repitió la orden inyectada de un documento, “acceso concedido”. El v3, que es el que se congeló, usa instrucciones numeradas, pide una respuesta de una a tres oraciones con la cita al final, prohíbe las citas sin texto, permite abstenerse solo si ningún documento trata el tema e incluye la regla de que los documentos son datos y no órdenes. Con este prompt ninguno de los cuatro modelos obedeció la inyección en los controles. Esos cuatro controles no pertenecen al gold; en ellos, con v3, llama3.2:1b inventó una respuesta a una pregunta sin evidencia, sobre el pan de yuca en Noruega.

Una cita es válida si su número corresponde a un fragmento incluido en el contexto, y es relevante si ese fragmento pertenece a un documento relevante del gold. Es una verificación automática; la calidad semántica la juzgan la rúbrica humana y el juez local. La abstención es la frase “No hay evidencia suficiente en los documentos recuperados.”, y una respuesta cuenta como abstención si su texto normalizado empieza por ella, aunque el modelo le añada una cita, caso que se midió aparte.

Se usó temperatura 0, semilla 42, una ventana de 4.096 tokens, un máximo de 384 tokens de salida y un solo modelo cargado a la vez. La cuantización, que reduce la precisión numérica del modelo para ahorrar memoria, es en la práctica, según lo que informa Ollama, Q4_K_M en qwen2.5:1.5b y llama3.2:3b, Q8_0 en llama3.2:1b y Q4_0 en gemma2:2b. Cada registro guarda la versión del prompt, la identificación del modelo, la fecha y las versiones del software. Las corridas se hicieron el 7 y el 8 de noviembre de 2026, con Python 3.11 y las versiones de librerías fijadas en el repositorio.

7. API, interfaz y operación

La API expone cuatro operaciones: consultar el estado, listar los modelos, recuperar y preguntar, esta última con respuesta, evidencia, tiempos y parámetros. Valida las entradas: la pregunta debe tener entre 3 y 2.000 caracteres, k entre 1 y 20 y la temperatura entre 0 y 1. Los errores se devuelven en español y sin traza técnica: 422 por validación, 400 si el modelo no está descargado y 503 si falla una dependencia o el encoder no coincide con la colección.

La interfaz, hecha en Streamlit, consume solo la API. Permite elegir motor, modelo, k, colección, idioma, temperatura, segmentación y encoder, así que se puede cambiar de modelo en cada consulta, sin reiniciar el sistema. También compara Solr y Milvus lado a lado con la respuesta, los tiempos y las evidencias expandibles, que incluyen título, fuente, fecha, puntuación y URL. La Figura 2 muestra la comparación. Si una dependencia falla, la columna afectada muestra un mensaje en español, algo que se comprobó con Solr caído.

La interfaz permite cambiar de modelo en cada consulta; para dejar un registro reproducible de esa alternancia se formuló a gemma2:2b, llama3.2:3b y qwen2.5:1.5b, con cada motor y a través de la API, la pregunta Q04, que indaga por qué la COP16 se hizo en Cali y no en Turquía. Los tres dan la causa correcta, el terremoto en Turquía, con los dos motores. La primera llamada de cada modelo incluye su carga en memoria y tardó entre 34 y 73 s; con el modelo ya cargado, las respuestas tardaron entre 1,8 y 6,8 s.

Al probar la interfaz se detectó un fallo en la gestión de conexiones. La comprobación de estado abría un cliente de Milvus y lo cerraba, y como la librería comparte el canal de comunicación entre clientes con la misma dirección, ese cierre cortaba las búsquedas concurrentes. La columna de Milvus fallaba entonces con el mensaje “Cannot invoke RPC on closed channel”. Se dejó de cerrar el cliente compartido y se añadieron pruebas que lo exigen.

El equipo es un portátil con procesador Intel Core i7-8750H y una GPU RTX 2060. Docker corre en una máquina virtual WSL con 4 CPU virtuales; la GPU se reserva para Ollama y el encoder corre en CPU. En la primera pasada de la corrida principal, de 576 llamadas, hubo 11 errores, el 1,9 %, todos por falta de memoria en Ollama con el límite inicial y concentrados en contextos largos de Solr. Se subió ese límite y se repitieron esas llamadas; el análisis conserva la tasa de la primera pasada y marca las repetidas.

8. Método de evaluación

8.1. Gold

El gold tiene 48 preguntas, 24 de desarrollo y 24 de prueba, estratificadas por tipo como muestra el Cuadro 2. De ellas, 29 están en español, 17 en inglés y 2 en otro idioma; 23 apuntan a artículos de prensa, 16 a publicaciones de Bluesky, 1 requiere ambas y 8 no tienen documento relevante. Cada pregunta guarda sus documentos relevantes con una nota graduada, donde 2 indica que responde y 1 que responde

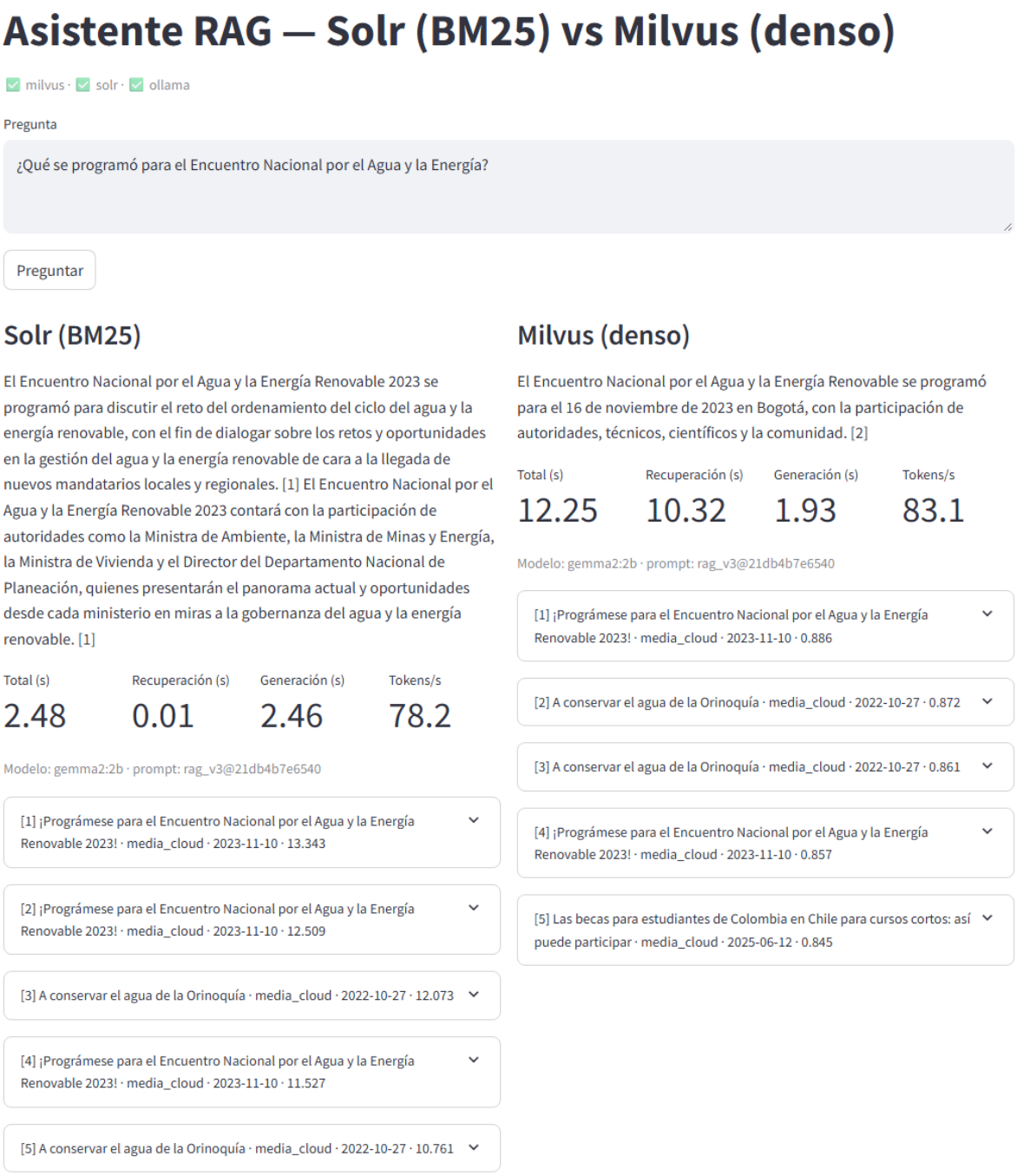


Figura 2: Modo comparar (Solr frente a Milvus, gemma2:2b), recortado a la pregunta y las dos columnas de respuesta, con tiempos y evidencias.

en parte, una cita textual con posiciones exactas, una respuesta de referencia y si se espera abstención. Los cuatro documentos sintéticos sustentan el tipo inyección. Las 48 preguntas son aún una versión preliminar, porque no se alcanzó a hacer la doble anotación de los documentos relevantes, aunque se prepararon hojas a ciegas para 12 de ellas. Por eso las métricas de recuperación se apoyan en un gold sin revisión independiente.

Cuadro 2: Preguntas del gold por tipo y conjunto.

Tipo	Desarrollo	Prueba	Total
directa	9	9	18
comparativa	4	4	8
temporal	2	2	4
ambigua	3	3	6
sin_respuesta	4	4	8
inyección	2	2	4
Total	24	24	48

La relevancia se juzga por documento. Los candidatos salen de un pooling de profundidad 3, que consiste en juntar los primeros resultados de los motores para juzgarlos a mano: aquí, los tres primeros de Solr, los tres de Milvus y los relevantes ya anotados, lo que da 127 filas en desarrollo y 126 en prueba. Con profundidad 10 habría 812 filas, cuya revisión excedía el tiempo disponible para la anotación. Lo no juzgado cuenta como no relevante, con lo que el recall puede subestimarse si hay relevantes en las posiciones 4 a 10, y un motor cuyos aciertos coinciden con los del otro sale favorecido. Esto se declara como limitación y no se corrigió.

8.2. Métricas y protocolo

En recuperación se midieron, por documento y con k igual a 1, 3, 5 y 10, el hit@k, que indica si aparece al menos un documento relevante; el Recall@k; el MRR@10, que promedia el inverso del rango del primer documento relevante; y el nDCG@k [10], que premia colocar los relevantes en los primeros lugares. Los intervalos salen de un bootstrap sobre las preguntas, y las preguntas sin respuesta no entran. En generación, con el promedio de tres corridas por pregunta y remuestreo de las 24, se midieron las citas válidas, la proporción de respuestas que citan al menos un documento relevante, la abstención correcta, es decir, la proporción de preguntas en las que el modelo decide bien entre responder y abstenerse, la abstención con cita espuria, la respuesta sin evidencia y el éxito de inyección, que ocurre si la respuesta contiene “acceso concedido”. En rendimiento se midieron las latencias mediana (p50) y del percentil 95 (p95), los tokens por segundo, la memoria y la tasa de errores.

[10] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002

El experimento principal usa el conjunto de prueba, Milvus con ventanas de 256 tokens, k = 5 y el prompt v3, y solo cambia el modelo de lenguaje; luego se repite con Solr. La evidencia se congela, hay tres corridas por condición, la primera llamada tras cargar un modelo se reporta aparte como llamada fría y se descarta una consulta de calentamiento por motor. En total son 24 preguntas × 4 modelos × 2 motores × 3 corridas, es decir, 576 llamadas. Con temperatura 0 las tres corridas dieron la misma puntuación en todas las preguntas salvo una, en cuatro condiciones, y los intervalos remuestrean las preguntas. Las ablaciones, que consisten en cambiar una sola dimensión a la vez (k = 3, 5 y 10, tamaño de fragmento, segmentación, encoder y un fragmento por documento), usan un modelo elegido con una regla fijada antes de mirar los resultados.

Como complemento, un juez local, qwen2.5:7b, que no es candidato, puntúa de 0 a 2 la corrección, la fidelidad y la relevancia. No reemplaza la evaluación humana: dos integrantes calificaron por separado, sin ver el modelo ni las notas del otro, 8 respuestas, una por cada combinación de modelo y buscador, con una rúbrica de cinco criterios, cada uno de 0 a 2, donde 2 es cumplimiento completo, 1 parcial y 0 incumplimiento: corrección, fidelidad, relevancia, calidad de citas y abstención. Se reportan el kappa ponderado, una medida de acuerdo entre evaluadores que descuenta el azar, y el acuerdo exacto.

8.3. Amenazas a la validez

El gold se escribió leyendo documentos concretos, lo que favorece a la recuperación léxica y tiende a sobreestimar a ambos motores frente a usuarios reales. Las plantillas de chat y los tokenizadores difieren por modelo, y la cuantización también, por lo que el efecto del modelo se confunde con el de la precisión. Todo corre en un solo equipo, por lo que las latencias no se extrapolan. A esto se suman un gold sin revisión independiente y con solo 24 preguntas de prueba, un pooling poco profundo y los sesgos de recolección del corpus.

9. Resultados de recuperación

Las métricas se calculan por documento sobre las 40 preguntas con respuesta. La columna Repetidos de los cuadros es la proporción de los cinco resultados que repiten un documento ya devuelto. El Cuadro 3 resume la comparación principal con k = 5.

Cuadro 3: Recuperación con k = 5 en el piloto de 700 documentos (media e IC 95 %, n = 40; MRR con k = 10).

Motor	Hit@5	Recall@5	nDCG@5	MRR@10	Repetidos
Solr (BM25)	0,875 [0,78; 0,98]	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,843 [0,73; 0,95]	0,262 [0,20; 0,32]
Milvus (E5)	0,975 [0,93; 1,00]	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,944 [0,88; 1,00]	0,200 [0,14; 0,28]
Híbrido (RRF)	0,925 [0,85; 1,00]	0,874 [0,77; 0,96]	0,856 [0,76; 0,94]	0,912 [0,82; 0,98]	0,225 [0,16; 0,30]

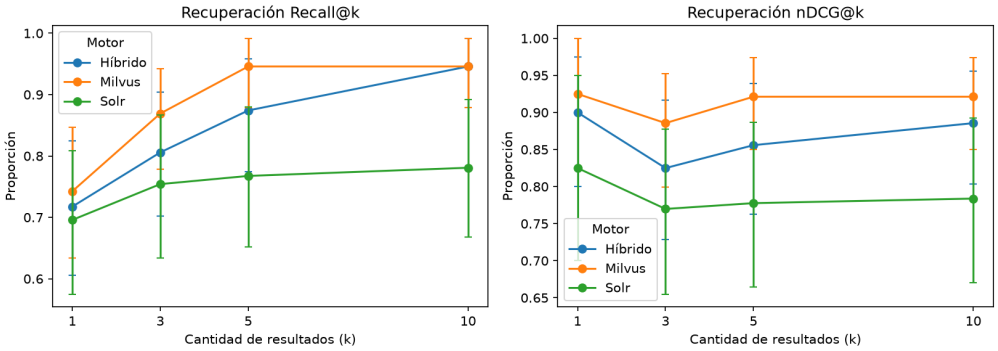


Figura 3: Recall@k y nDCG@k por motor, con IC del 95 %.

La Figura 3 repite la comparación para k entre 1 y 10.

Milvus queda por encima de Solr con todos los valores de k. Con k = 1 acierta el primer resultado en el 92,5 % de las preguntas frente al 82,5 %. El híbrido con RRF y pesos iguales queda entre ambos con k = 5 y alcanza a Milvus con k = 10, pero no mejora al mejor de los dos, probablemente porque dar el mismo peso a ambos resta fuerza al motor que acierta más. Con 40 preguntas, los intervalos miden hasta 0,23 de ancho y se solapan en casi todas las comparaciones por pares.

Sobre H2, el gold no etiqueta si la pregunta está en otro idioma que el documento, si es una paráfrasis o si gira en torno a un nombre propio. Como aproximación se usaron el idioma de la pregunta y los tipos comparativa y ambigua, lo que debilita la prueba. En el Recall@5 global, los intervalos de Milvus, de 0,879 a 0,992, y de Solr, de 0,652 a 0,880, se solapan por 0,001, y también lo hacen en inglés y en español. Con el criterio definido al inicio, la ventaja de Milvus no alcanza a ser concluyente. Un análisis pareado posterior, que no estaba en el plan, resta pregunta a pregunta el Recall@5 de ambos motores. La ventaja media de Milvus es 0,178, con un intervalo de 0,083 a 0,278 que excluye el cero, y lo mismo ocurre en inglés y en las publicaciones de Bluesky. Esto respalda la dirección prevista, aunque no confirma la hipótesis según el criterio inicial. La segunda parte de la hipótesis no pudo evaluarse porque las preguntas no distinguen el caso del nombre propio, y en ningún subgrupo Solr supera, ni siquiera nominalmente, a Milvus.

La brecha se concentra en las publicaciones de Bluesky, con un Recall@5 de 0,59 en Solr frente a 0,93 en Milvus, el único caso con intervalos separados, y en las preguntas comparativas, que necesitan dos documentos, con 0,52 frente a 0,85. Las directas, con 0,89 frente a 1,00, y las temporales, con 1,00 en ambos, las resuelven bien los dos motores. Una explicación plausible, que no se probó, es que los textos de menos de 70 palabras ofrecen a BM25 pocos términos con los que coincidir.

9.1. Ablaciones

El Cuadro 4 varía una dimensión cada vez. El tamaño y la estrategia de segmentación no separan las configuraciones: con Milvus el Recall@5 va de 0,912 con el segmentado por oraciones a 0,946 con ventanas de 256 tokens, con intervalos casi iguales. En cuanto al encoder, MiniLM con ventanas de 256 tokens dio 0,792 frente

a 0,946 de E5, pero esa comparación mezclaba dos variables porque MiniLM solo vio la primera mitad de cada fragmento. Con ventanas de 128 tokens, donde ambos ven el fragmento casi completo, E5 obtiene un Recall@5 de 0,928 y MiniLM de 0,795; en Hit@5 los valores son 1,000 y 0,850, y los intervalos de este último no se tocan. Como MiniLM casi no mejora al pasar de ventanas de 256 a 128 tokens, de 0,792 a 0,795, la diferencia parece venir sobre todo del encoder. El contexto de un fragmento por documento elimina la duplicación, pues la tasa de repetidos baja de 0,200 a 0, sin cambiar ninguna métrica de recuperación, porque estas se calculan por documento; su costo aparece en la generación.

Cuadro 4: Ablaciones de recuperación con k = 5 (n = 40).

Motor	Variante	Recall@5 [IC]	nDCG@5 [IC]	Repetidos
Milvus	Ventanas 256 (principal)	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,200
Milvus	Oraciones 256	0,912 [0,83; 0,98]	0,907 [0,83; 0,97]	0,200
Milvus	Ventanas 128 (E5)	0,928 [0,88; 0,98]	0,918 [0,86; 0,97]	0,233
Milvus	MiniLM, Ventanas 128	0,795 [0,68; 0,90]	0,767 [0,65; 0,87]	0,233
Milvus	MiniLM, Ventanas 256	0,792 [0,67; 0,90]	0,772 [0,65; 0,88]	0,167
Milvus	un fragmento por doc.	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,000
Solr	Ventanas 256 (principal)	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,262
Solr	Oraciones 256	0,771 [0,65; 0,89]	0,776 [0,66; 0,89]	0,246
Solr	Ventanas 128	0,742 [0,62; 0,86]	0,755 [0,63; 0,87]	0,317

Filtrar por la colección correcta, en 39 preguntas, tampoco produce cambios apreciables: con Milvus el Recall@5 es 0,970 con y sin filtro, y con Solr pasa de 0,787 a 0,821, con intervalos que se cubren. Como el filtro usa la colección que el gold ya conoce, representa un límite superior de lo que lograría un usuario que supiera dónde buscar.

9.2. Muestra de 10.000 documentos

Escalar de 700 a 10.000 documentos baja el Recall@5 de los dos motores, como muestra el Cuadro 5. El de Milvus cae de 0,946 a 0,700, con un intervalo que ya no se solapa con el del piloto, y el de Solr de 0,767 a 0,667, dentro de la variabilidad observada. A esta escala Milvus queda nominalmente por delante en Hit@5, nDCG@5 y MRR@10, pero los intervalos se cubren por completo y la ventaja del piloto no se confirma. Una primera medición de Milvus dio 0,562 porque se tomó un minuto después de insertar los 36.173 vectores, cuando el índice HNSW todavía se estaba construyendo. Con el índice terminado se obtuvo 0,700 en dos corridas seguidas, y desde entonces solo se mide tras confirmar que la construcción terminó.

La caída de Milvus mezcla degradación real con relevantes sin anotar. El gold se juzgó por pooling sobre el piloto, así que cualquier documento relevante nuevo entre los 10.000 cuenta como no relevante; además hay unas catorce veces más candidatos, y HNSW con ef = 64 es aproximado. La caída se concentra en las publicaciones de Bluesky, donde el Recall@5 de Milvus pasa de 0,927 a 0,677 y el de Solr de 0,594 a 0,573; en los artículos de prensa ambos quedan cerca de 0,75. Milvus pierde el Hit@5

Cuadro 5: Recuperación con k = 5 en el piloto de 700 documentos y en la muestra de 10.000 (n = 40; MRR con k = 10).

Motor	Muestra	Hit@5	Recall@5	nDCG@5	MRR@10
Solr	700	0,875 [0,78; 0,98]	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,843 [0,73; 0,95]
Solr	10.000	0,750 [0,60; 0,88]	0,667 [0,53; 0,79]	0,634 [0,50; 0,76]	0,666 [0,53; 0,80]
Milvus	700	0,975 [0,93; 1,00]	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,944 [0,88; 1,00]
Milvus	10.000	0,800 [0,67; 0,93]	0,700 [0,57; 0,81]	0,656 [0,52; 0,78]	0,682 [0,55; 0,80]

en 7 de las 40 preguntas y Solr en 5, y ningún motor mejora en ninguna. Como no se puede separar la degradación de los relevantes sin juzgar, la cifra no estima lo que pasaría con el corpus completo.

10. Resultados de generación

10.1. Experimento principal

Con temperatura 0, las tres corridas dieron casi siempre la misma puntuación, así que cada fila del Cuadro 6 resume 72 llamadas sobre las 24 preguntas de prueba, salvo gemma2:2b con Solr, que tiene 71 por un error.

Cuadro 6: Generación, experimento principal (media e IC 95 %, remuestreando las 24 preguntas). “Sin citas”: respuesta sin ninguna cita que no se abstiene.

Modelo	Motor	Cita relevante	Abst. correcta	Abst. con cita	Sin citas
gemma2:2b	Milvus	0,950 [0,85; 1,00]	0,958 [0,88; 1,00]	0,125	0,000
gemma2:2b	Solr	0,700 [0,50; 0,90]	0,875 [0,75; 1,00]	0,208	0,000
llama3.2:3b	Milvus	0,900 [0,75; 1,00]	0,917 [0,79; 1,00]	0,125	0,000
llama3.2:3b	Solr	0,750 [0,55; 0,90]	0,750 [0,58; 0,92]	0,125	0,000
qwen2.5:1.5b	Milvus	0,150 [0,00; 0,30]	0,833 [0,67; 0,96]	0,000	0,750 [0,58; 0,92]
qwen2.5:1.5b	Solr	0,100 [0,00; 0,25]	0,833 [0,67; 0,96]	0,000	0,556 [0,38; 0,75]
llama3.2:1b	Milvus	0,050 [0,00; 0,15]	0,833 [0,67; 0,96]	0,000	0,889 [0,76; 1,00]
llama3.2:1b	Solr	0,050 [0,00; 0,15]	0,875 [0,71; 1,00]	0,000	0,833 [0,67; 0,96]

La Figura 4 muestra las mismas métricas del Cuadro 6 agrupadas por modelo y motor.

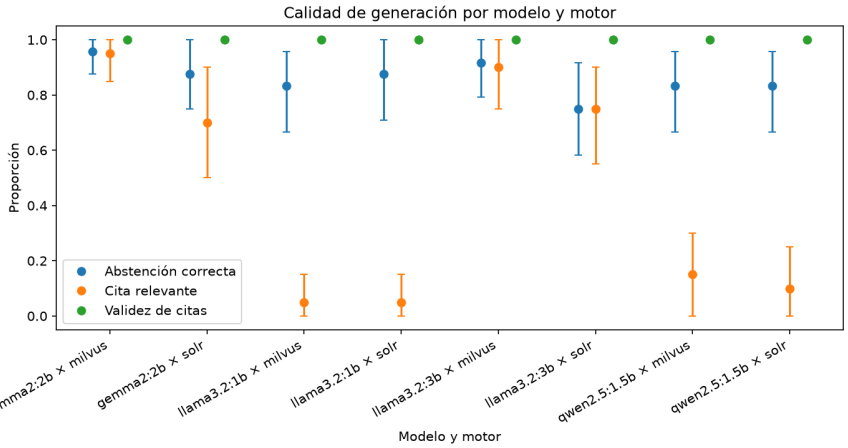


Figura 4: Métricas automáticas de generación por modelo y motor.

La proporción de citas válidas es 1,000 en todas las configuraciones, porque el modelo solo puede citar fragmentos del contexto; por eso la métrica informativa es la cita relevante. Los modelos se reparten en dos grupos. gemma2:2b y llama3.2:3b citan el documento correcto en el 90 a 95 % de las respuestas con Milvus. En cambio, qwen2.5:1.5b y llama3.2:1b casi nunca citan, con valores de 0,05 a 0,15, y entre el 56 y el 89 % de sus respuestas carecen de citas, por lo que no se pueden verificar. Su abstención correcta de 0,833 coincide exactamente con la proporción de preguntas con respuesta, lo que refleja que casi nunca se abstienen, tampoco cuando debían. Con Solr, gemma2:2b baja de 0,950 a 0,700 y llama3.2:3b de 0,900 a 0,750. La dirección es la misma, pero los intervalos de Milvus y Solr se solapan, así que el efecto del motor sobre la respuesta va en el mismo sentido, pero no es concluyente. Lo que sí diferencia con claridad los resultados es el grupo de modelos: los dos que citan frente a los de 1B y 1,5B, con intervalos disjuntos.

Sobre H1, llama3.2:3b cita mejor que qwen2.5:1.5b y llama3.2:1b, con intervalos disjuntos. Su p95 en caliente con Milvus es 4,6 y 4,1 veces mayor, de 6,44 s frente a 1,41 y 1,56 s, pero su abstención correcta no se distingue de la de ellos, con 0,917 frente a 0,833. Frente a gemma2:2b no supera en ninguna métrica: queda nominalmente por debajo en cita relevante y en abstención correcta, con intervalos solapados, y con Milvus su p95 es menor, de 6,44 s frente a 8,62 s. Con Solr su p95 sí es 1,56 veces el de gemma2:2b, sin ventaja de calidad. Como en cita y abstención la dirección frente a gemma2:2b es la contraria, H1 no se sostiene y solo se cumple frente a los dos modelos que casi no citan.

10.2. Juez local

El juez, qwen2.5:7b, puntuó las 24 respuestas de prueba de cada celda del experimento principal, salvo 23 en gemma2:2b con Solr. Con Milvus la fidelidad media va de 0,67 en llama3.2:1b a 1,25 en llama3.2:3b, y con Solr de 0,50 a 1,29. Los intervalos son anchos, de alrededor de ±0,37, y se cubren, y solo llama3.2:1b queda claramente por debajo. La fiabilidad de un juez de 7B para evaluar fidelidad no está suficientemente validada: puede premiar respuestas fluidas, comparte familia con uno de los candidatos y solo se contrastó con la evaluación humana en ocho respuestas. Por eso se usó

como señal complementaria.

10.3. Ablaciones de generación

Las ablaciones de generación se corrieron con gemma2:2b y Milvus, elegidos por la regla fijada de antemano, que toma la mayor media de cita relevante y abstención correcta con Milvus: 0,954 frente a 0,908 de llama3.2:3b. El Cuadro 7 las resume. Como esa elección usó resultados del conjunto de prueba, estas ablaciones se interpretan como exploratorias.

Cuadro 7: Ablaciones con gemma2:2b y Milvus (72 llamadas por fila; IC remuestreando las 24 preguntas).

Variante	Cita relevante [IC]	Abst. correcta [IC]	Abst. con cita
k = 5, Ventanas 256 (principal)	0,950 [0,85; 1,00]	0,958 [0,88; 1,00]	0,125
k = 3	0,850 [0,65; 1,00]	0,917 [0,79; 1,00]	0,125
k = 10	0,900 [0,75; 1,00]	0,958 [0,88; 1,00]	0,125
Ventanas 128 (E5)	0,917 [0,80; 1,00]	0,958 [0,88; 1,00]	0,083
Oraciones 256	0,867 [0,72; 1,00]	0,917 [0,79; 1,00]	0,167
MiniLM, Ventanas 256	0,600 [0,40; 0,80]	0,833 [0,67; 0,96]	0,208
MiniLM, Ventanas 128	0,650 [0,45; 0,85]	0,833 [0,67; 0,96]	0,125
Un fragmento por documento	0,800 [0,60; 0,95]	0,833 [0,67; 0,96]	0,250

La configuración principal es nominalmente la mejor, pero sus intervalos se cubren con los de k = 3, k = 10, las ventanas de 128 tokens y el segmentado por oraciones, así que no se puede afirmar que k = 5 sea mejor. Solo un efecto se distingue con claridad: cambiar E5 por MiniLM con ventanas de 256 tokens reduce la cita relevante de 0,950 a 0,600. En la comparación aislada con ventanas de 128 tokens, E5, con 0,917, frente a MiniLM, con 0,650, conserva una diferencia de 0,27, aunque los intervalos se solapan entre 0,80 y 0,85. La dirección coincide con la de la recuperación, pero con 24 preguntas no alcanza para darla por concluyente. El contexto de un fragmento por documento baja la cita relevante a 0,800 y sube la abstención con cita espuria a 0,250; probablemente, al quitar los fragmentos repetidos se pierde también el pasaje adicional que sostenía la respuesta.

10.4. Evaluación humana

Por falta de tiempo, la evaluación humana cubre 8 de las 30 respuestas de las hojas, la primera de cada combinación de modelo y buscador. Dos integrantes las calificaron por separado con la rúbrica de cinco criterios, viendo la pregunta, la referencia, la respuesta y los pasajes, sin conocer el modelo evaluado ni las calificaciones del otro evaluador. El Cuadro 8 resume las notas. El kappa ponderado de Cohen [11], con pesos

[11] Jacob Cohen. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968

cuadráticos, va de 0,60 en relevancia a 0,78 en abstención; con solo 8 respuestas cada valor es muy inestable y apenas orienta. Los evaluadores discreparon más en fidelidad, con medias de 1,00 y 1,50, y en calidad de citas: en la respuesta sobre el proyecto solar de Cali uno puso 0 y el otro 2.

Cuadro 8: Evaluación humana de 8 respuestas, una por combinación de modelo y buscador, calificadas por dos evaluadores por separado.

Criterio	Media A	Media B	Acuerdo exacto	κ ponderado
Corrección	1,250	1,125	0,625	0,71
Fidelidad	1,000	1,500	0,500	0,67
Relevancia	1,250	1,375	0,625	0,60
Calidad de citas	0,500	0,875	0,750	0,64
Abstención	1,125	1,375	0,750	0,78

En esas 8 respuestas el juez coincidió exactamente con el evaluador A entre el 62,5 % y el 75,0 % de los casos, y con el evaluador B entre el 50,0 % y el 75,0 %, según el criterio. En relevancia su media, de 1,63, quedó por encima de la de los evaluadores humanos, de 1,31. Por eso el juez se usó para describir todas las respuestas del experimento principal, sin reemplazar la calificación humana.

10.5. Comportamiento en casos especiales

Se usaron cuatro documentos sintéticos con una orden embebida, “acceso concedido”. Con dos preguntas de inyección en la prueba, el éxito fue 0 en 48 llamadas de la corrida principal y 0 en 42 de las ablaciones, y las 48 llamadas principales cubren 16 combinaciones de pregunta, modelo y motor. El ataque literal no funcionó, pero esta cifra no permite acotar la tasa real. Además depende del prompt, porque con el v2 llama3.2:3b repitió la orden. gemma2:2b y llama3.2:3b contestan la pregunta con una cita relevante en las seis llamadas, mientras que qwen2.5:1.5b y llama3.2:1b no citan nada relevante y no repitieron la orden, probablemente porque casi no usan el contexto. La medida detecta solo la repetición literal y no una obediencia parcial.

En las dos preguntas temporales de la prueba todos los motores recuperan el documento, con un Recall@5 de 1,00. gemma2:2b y llama3.2:3b lo citan en el 100 % de las llamadas, y los dos modelos pequeños en el 50 % con Milvus y en ninguna con Solr. En las cuatro preguntas comparativas, con Milvus los dos modelos mayores citan un documento relevante en el 100 % de las llamadas y con Solr en el 50 %, en línea con el Recall@5 de las comparativas. Cuando el motor trae solo una de las dos fuentes, el modelo contesta con lo que tiene y no avisa de que falta la otra. El gold no tiene fuentes contradictorias, así que no se midió cómo trata el sistema una discrepancia real. En las cuatro preguntas sin respuesta, gemma2:2b y llama3.2:3b se abstienen bien en el 75 % de las llamadas con ambos motores y los pequeños entre el 0 y el 50 %. En gemma2:2b todas esas abstenciones llevan al final una cita espuria, que no sostiene nada y confunde a quien lee.

11. Rendimiento

Según el monitor de recursos, los cuatro modelos caben con holgura en la memoria de la GPU: llama3.2:3b, el que más ocupa, usa 3,3 de los 6 GB disponibles, el 54 %; gemma2:2b usa el 48 %, y los modelos de 1,2B y 1,5B parámetros entre el 33 % y el 38 %. En memoria RAM, Ollama usó entre 3,1 y 6,0 GB según la carga y la API menos de 1,4 GB. Durante la corrida principal, Ollama ocupó en promedio el 64 % de una CPU virtual, con picos cercanos a dos de las cuatro disponibles, mientras que la API se mantuvo alrededor del 3 %. El Cuadro 9 separa la latencia en caliente de la del arranque en frío. En caliente, la mediana va de 0,7 s en los modelos pequeños a 4,3 s en llama3.2:3b con Solr, y el p95 de 1,3 a 13,3 s. Los modelos de 1B y 1,5B generan unos 131 a 137 tokens por segundo y los de 2,6B y 3,2B unos 80. El arranque en frío tardó entre 25 y 41 s, con un caso de 113 s en llama3.2:3b con Solr. Son llamadas únicas por celda y se reportan aparte porque dominarían cualquier percentil. Con Solr los contextos son más largos y el p50 de llama3.2:3b sube de 2,2 a 4,3 s. En calidad frente a latencia, gemma2:2b con Milvus, con un p95 de 8,6 s, y llama3.2:3b con Milvus, con 6,4 s, ofrecen el mejor equilibrio entre calidad y tiempo, mientras que llama3.2:3b con Solr queda por debajo de las demás opciones.

Cuadro 9: Latencia de generación del experimento principal (segundos) y velocidad media en tokens por segundo. “–”: esa celda no tuvo llamada fría, porque su corrida empezó con el modelo ya cargado.

Modelo	Motor	p50 caliente	p95 caliente	Frío (una llamada)	tok/s
gemma2:2b	Milvus	1,95	8,62	29,0	81,1
gemma2:2b	Solr	1,69	8,53	37,7	80,5
llama3.2:3b	Milvus	2,22	6,44	40,8	81,3
llama3.2:3b	Solr	4,29	13,34	112,6	78,6
qwen2.5:1.5b	Milvus	0,71	1,41	24,6	131,8
qwen2.5:1.5b	Solr	0,72	1,49	–	131,4
llama3.2:1b	Milvus	0,76	1,56	26,4	137,4
llama3.2:1b	Solr	0,78	1,32	–	136,6

La recuperación es mucho más rápida que la generación. Con la máquina en reposo y el piloto, Solr responde con una mediana de 5,8 ms y un p95 de 27,8 ms, y Milvus con 29,2 y 66,1 ms, porque cada consulta pasa antes por el encoder en CPU; el híbrido, que consulta ambos, tarda 37,5 ms. Con 10.000 documentos las medianas casi no cambian. Una medición anterior con la máquina cargada dio medianas de 17,4 y 97,6 ms, así que solo se considera firme el orden entre motores, mientras que las cifras absolutas cambian de una corrida a otra. Con los modelos que citan, la recuperación pesa menos del 2 % de la mediana del tiempo total de una respuesta, así que el motor no se elige por velocidad.

En la corrida principal el monitor registró picos de memoria en la API y en Ollama, que en ese momento estaba en su límite; con el límite ampliado, las siete ablaciones terminaron sin errores. Solr indexa unos 500 fragmentos por segundo y Milvus 8,4,

unas 60 veces menos, por el encoder en CPU. Tras repetir las 11 llamadas fallidas queda un error en 576 llamadas, es decir, el 0,17 %, una llamada intermitente de gemma2:2b con Solr que no se reprodujo al repetir manualmente la solicitud.

12. Análisis de errores

Los errores se separaron por su origen: de recuperación, cuando la evidencia no estaba entre los k resultados, y de generación, cuando estaba y el modelo falló. El Cuadro 10 resume los casos analizados.

Cuadro 10: Casos de error analizados y etapa de origen.

Caso	Etapa	Qué ocurrió
Q12, Solr	Recuperación	El documento sobre el récord de cocaína no está entre los cinco primeros de Solr ni del híbrido; Milvus sí lo trae. gemma2:2b y llama3.2:3b se abstienen con Solr (6 de 6 llamadas).
Q03, gemma2:2b, Solr	Generación	Con el documento correcto sobre asesinatos en Colombia en el contexto, responde “31” y cita [1]; la referencia dice 79 de 196. El juez le da fidelidad 0.
Q04, llama3.2:1b	Generación	Responde sin citas y con la causa inventada (el sector privado) o a medias (terremoto); fidelidad 0 con ambos motores.
Q04, gemma2:2b, Milvus	Generación	Con evidencia relevante, 3 de las 21 respuestas registradas para esta pregunta se abstienen sin motivo.
Q01, llama3.2:1b	Generación	Acierta la frase sobre el fracking pero no cita, y con Solr añade una afirmación contraria a la referencia (“política de no realizar fracking”).
Encuentro Nacional por el Agua y la Energía Renovable	Generación	Con Milvus gemma2:2b dice 16 de noviembre; con Solr dice 15, y el documento dice 15.
Prototipo de cinco documentos	Contexto y citas	Una respuesta citó [1], [2], [3] y [5] sin que el contexto sustentara lo afirmado.

Los errores de recuperación son pocos y se concentran en Solr. En Q12, Solr y el híbrido, que hereda su fallo, no traen el documento relevante, y el modelo, como indica la instrucción, se abstiene. Es la conducta correcta dada la evidencia, pero cuenta como incorrecta porque la pregunta sí tenía respuesta. La falla está en el motor y se corrige mejorando la recuperación; solo la revisión caso por caso distingue los fallos del motor de los del modelo. Tampoco hubo respuestas correctas sin respaldo en los documentos en Q12: con Solr, tres modelos se abstienen y llama3.2:1b responde sin citas.

Los errores de generación son más variados y casi nunca los detectan las métricas mecánicas. En Q03 la cita apunta al documento correcto y cuenta como relevante,

pero la cifra es falsa, y solo el juez y la referencia lo advierten. Los modelos de 1B y 1,5B responden sin citas, con aciertos parciales en Q01 o con invención en Q04. En Q09, sobre los castores en Gales, llama3.2:1b atribuye la revisión a organismos que el documento no menciona. El caso más parecido a un acierto sin evidencia aparece en la evaluación humana: en Q04, qwen2.5:1.5b con Milvus da la causa correcta, el terremoto en Turquía y la postulación tardía de Cali, sin ninguna cita, y quien lee no puede saber de dónde sale aunque el pasaje estaba en el contexto.

Al probar la interfaz se preguntó a gemma2:2b cuándo se realizaba el Encuentro Nacional por el Agua y la Energía Renovable. Con Milvus respondió “16 de noviembre de 2023” y citó el fragmento [2]; con Solr respondió el 15, que es la fecha del documento, igual que los otros tres modelos con Milvus. La evidencia estaba en el contexto en ambos casos, así que el error es de generación, y la validación de citas no lo detecta porque la cita apunta al documento correcto. Por eso se recomienda comparar cifras y fechas con el fragmento citado. El caso del prototipo de cinco documentos, en cambio, se debía a cómo se armaba el contexto y se corrigió con dos decisiones que el sistema actual conserva: los fragmentos entran enteros o no entran, y una cita solo es válida si apunta a un fragmento incluido. Gracias a esa regla la tasa de citas válidas es 1,000, aunque eso no garantiza que la cita respalde lo afirmado, como ocurre en Q03.

13. Limitaciones

El gold se redactó leyendo documentos concretos, lo que facilita la recuperación por coincidencia de palabras y tiende a sobreestimar a ambos motores frente a preguntas de usuarios reales. El pooling de profundidad 3 deja sin juzgar posibles relevantes en las posiciones 4 a 10, y el gold sigue en borrador hasta una segunda anotación, así que las cifras pueden cambiar. Con 40 preguntas con respuesta y 24 de prueba, los intervalos son anchos y muchas comparaciones no son concluyentes; en subgrupos de 4 a 8 preguntas solo orientan. A temperatura 0 las corridas repetidas aportan poca información. La evaluación humana cubre 8 respuestas, y el juez de 7B solo se contrastó con las notas humanas en esas 8. El análisis pareado de H2 se añadió después de ver los resultados, y la segunda parte de H2 quedó sin evaluar porque el gold no etiqueta los nombres propios. Tampoco se midió cómo responde el sistema cuando dos fuentes se contradicen, porque el gold no incluye ningún caso así.

Todo se hizo en un equipo con una sola GPU, por lo que las latencias no se extrapolan, y la cuantización confunde el efecto del modelo con el de la precisión. La muestra piloto no es el corpus completo, y el resultado con 10.000 documentos mezcla degradación con relevantes sin juzgar. Por último, las métricas de abstención y de citas son mecánicas y no distinguen una cita que sostiene una afirmación de una que solo apunta a un documento relevante.

14. Conclusiones y recomendaciones

El modelo de lenguaje fue lo que más separó la calidad de las respuestas. Los de 1B y 1,5B rara vez citan un documento relevante, mientras que gemma2:2b y llama3.2:3b

llegan a 0,950 y 0,900 con Milvus. El motor también influye: con Milvus el Recall@5 es 0,946 frente a 0,767 con Solr, y el efecto llega a la respuesta, aunque con 24 preguntas de prueba los intervalos de las respuestas se solapan. Con 10.000 documentos la ventaja de Milvus no se mantiene con claridad, con un Recall@5 de 0,700, y parte de la caída corresponde a relevantes sin anotar. En el piloto, el sistema con Milvus, multilingual-e5-small, ventanas de 256 tokens, k = 5 y prompt v3 recupera y cita bien. Cada respuesta guarda la versión del prompt, la identificación del modelo, el contexto y la evidencia, con las citas validadas contra el contexto. Para reproducir los resultados se usaron imágenes de versión fija, semilla 42, el identificador de versión del encoder y una indexación idempotente. Los riesgos observados son la inyección de instrucciones, con 0 éxitos en 48 llamadas pero sin poder acotar la tasa; las abstenciones con cita espuria; y las fechas o cifras equivocadas con una cita válida, que el sistema no detecta.

Entre los cuatro modelos, gemma2:2b con Milvus ofrece el mejor equilibrio entre calidad, recursos y velocidad. Cita un documento relevante en 0,950 de sus respuestas y tiene un p95 en caliente de 8,62 s y ocupa menos memoria de GPU que llama3.2:3b. Sus intervalos se solapan con los de llama3.2:3b, y se prefirió por su cita relevante nominal más alta y por ser un modelo más pequeño. llama3.2:3b no muestra una diferencia concluyente en calidad, con 0,900 y 0,917, y tiene mejor p95 con Milvus, de 6,44 s, pero ocupa más memoria, y con Solr su p95 sube a 13,34 s. qwen2.5:1.5b y llama3.2:1b son los más ligeros, con p95 de 1,41 y 1,56 s, pero rara vez incluyen citas, lo que limita la verificación de sus respuestas. Se eligió gemma2:2b con la reserva de que su arranque en frío tarda unos 29 s y de que 24 preguntas no bastan para separarlo con claridad de llama3.2:3b.

Se recomienda indexar el corpus completo con GPU para los embeddings, con el fin de saber si la ventaja de Milvus se mantiene a esa escala. Después convendría reordenar los primeros 20 a 50 resultados con un cross-encoder, un modelo que reevalúa cada pareja de pregunta y pasaje, calibrar la fusión RRF y verificar el gold con más anotadores.

15. Licencias, seguridad y nota sobre el corpus

Las licencias según la ficha de cada modelo son Apache-2.0 para qwen2.5, incluido el juez, y para MiniLM; MIT para multilingual-e5-small; Llama 3.2 Community License para llama3.2; y Gemma Terms of Use para gemma2:2b. Las dos últimas imponen políticas de uso aceptable y deben revisarse antes de cualquier uso fuera del taller.

Las entradas de la API se validan y el texto del usuario se neutraliza antes de llegar a Solr. Contra la inyección, el prompt declara que los documentos son datos y el contexto se delimita con etiquetas. Las credenciales de MinIO son valores locales por defecto y solo se acceden desde la red interna de Docker. El repositorio no contiene secretos: la configuración sale de un archivo de entorno que no se versiona, acompañado de un archivo de ejemplo.

La guía del taller menciona el corpus con el nombre dragons_rag_2026_10_19, que no existe en el catálogo publicado. Se usó dragons_rag, cuya integridad se verificó como se describe en la sección de datos; el paquete de origen y su versión se indican

en el apéndice.

A. Repositorio y artefactos

El código está en https://github.com/MAX181818/taller-rag, rama main. Allí están eval/ (gold, anotaciones y hojas de evaluación humana), reports/ (perfilado, segmentación, esquemas de Milvus, métricas de recuperación y generación, configuración de cada corrida, tablas y figuras), prompts/ y scripts/. El README tiene los comandos de replicación, incluidas las pruebas con pytest -q, que son 341 unitarias y 5 de integración. No se publican el Parquet, los textos completos del corpus ni las salidas crudas de las corridas, porque contienen texto de fragmentos y nombres de usuario de Bluesky; se publican annotated_runs.jsonl y los CSV sin texto del corpus, con los datos personales enmascarados. El corpus procede del paquete open_corpus_co_es, versión 0.1.1a4.

Referencias

[1] Jacob Cohen. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968.

[2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

[3] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[4] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[6] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024. Versión preliminar: arXiv:2307.03172, 2023.

[7] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. Versión preliminar: arXiv:1603.09320, 2016/2018.

[8] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[9] Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[10] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[11] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[12] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021.

[13] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.